AWS DynamoDB — Query & Transaction Fundamentals

Simple revision guide based on the NextWork “Query Data with DynamoDB” project

Goal: Quickly refresh the fundamental concepts: what they are, their purpose, why we use them, and a simple example.

1. Amazon DynamoDB

What is it? A non-relational (NoSQL) database that stores data in tables, items, and attributes.

Purpose: Store and retrieve data without the fixed row-and-column structure of a traditional relational database.

Why use it? It is flexible, provides fast access through keys, can scale, and supports efficient queries.

Simple example: A content app can store projects and videos in the same table even when they have different attributes.

2. DynamoDB Table

What is it? A container where DynamoDB organizes related data.

Purpose: Keep a collection of related items together.

Why use it? It gives the database a defined place to store and query data.

Simple example: A **ContentCatalog** table can store information about projects and videos.

3. Item

What is it? A single record/object stored in a DynamoDB table.

Purpose: Represent one thing in your application.

Why use it? Each item can contain its own attributes, making DynamoDB flexible.

Simple example: One item can represent one video, while another represents one project.

4. Attribute

What is it? A piece of data that describes an item.

Purpose: Store details about an item.

Why use it? Different DynamoDB items can have different sets of attributes.

Simple example: A video might have **Title**, **ContentType**, and **URL**, while a project can also have **Difficulty**.

5. Partition Key

What is it? The key DynamoDB uses to organize and efficiently find items in a table. Every item needs a value for it.

Purpose: Identify or locate the relevant data during a query.

Why use it? Queries depend on the partition key, so choosing it correctly is a major part of DynamoDB data modeling.

Simple example: If **Id** is the partition key, querying **Id = 201** can find that item.

6. Sort Key

What is it? An optional key used with a partition key to further organize or filter related items.

Purpose: Help distinguish and retrieve multiple related items that share a partition key.

Why use it? It lets you query related data more precisely, such as comments by date.

Simple example: In the **Comment** table, **Id** identifies the post and **CommentDateTime** can filter comments after a date.

7. Primary Key

What is it? The key structure DynamoDB uses to uniquely identify an item. With a composite key, the partition key and sort key form the combination.

Purpose: Uniquely identify items and organize related records.

Why use it? It allows multiple items to share a partition key while remaining distinguishable through the combination.

Simple example: Several comments can share the same post ID while different timestamps distinguish the comments.

8. Data Modeling

What is it? Planning how DynamoDB tables, partition keys, sort keys, and data structures should be designed before loading data.

Purpose: Make the queries your application needs easy and efficient.

Why use it? Poor key design can make some searches slow or difficult with a DynamoDB query.

Simple example: If you need comments for a post by date, design the keys around the post and comment time.

9. Query

What is it? A way to retrieve specific data from DynamoDB using the table's key structure.

Purpose: Find the items you need efficiently.

Why use it? Queries work best when you provide the required partition key and, when applicable, a sort key condition.

Simple example: Query **ContentCatalog** with **Id = 201**.

10. AWS CloudShell

What is it? A shell/terminal inside the AWS Management Console with AWS CLI already installed.

Purpose: Provide a ready-to-use command-line environment for managing AWS resources.

Why use it? You can run AWS commands without installing the AWS CLI on your own computer.

Simple example: Open CloudShell and run an **aws dynamodb** command.

11. AWS CLI

What is it? A command-line tool for creating, updating, deleting, and interacting with AWS resources.

Purpose: Manage AWS services using commands instead of clicking through the console.

Why use it? It is fast, repeatable, scriptable, and useful for automation.

Simple example: Use **aws dynamodb create-table** to create a table.

12. get-item

What is it? An AWS CLI DynamoDB command used to retrieve a single item using its key.

Purpose: Get one specific item from a table.

Why use it? It is useful when you already know the item's primary key.

Simple example: Retrieve the item whose **Id** is **201**.

CLI Query Options

Option	Simple meaning	Why use it?
--consistent-read	Request a strongly consistent read.	Get the guaranteed most recent version when freshness matters.
--projection-expression	Choose which attributes to return.	Return only the fields you need.
--return-consumed-capacity TOTAL	Show consumed read capacity.	See how much capacity the request used.

13. Strongly Consistent vs Eventually Consistent Reads

Strongly consistent: returns the guaranteed most recent version of an item.

Eventually consistent: the default read option in the project; it is lower cost/faster, but the returned version might not be the most updated immediately after a change.

> **Example:** Choose strong consistency when the latest value matters; eventual consistency is suitable when a small delay is acceptable.

14. Batch Write Item

What is it? An AWS CLI operation used to load or insert multiple items into DynamoDB.

Purpose: Load many records efficiently instead of inserting them one by one.

Why use it? It saves time when importing prepared JSON datasets.

> **Simple example:** Use **batch-write-item** with a JSON request file.

15. JSON Data File

What is it? A structured text file containing data formatted for DynamoDB operations.

Purpose: Provide items and attributes to AWS CLI commands.

Why use it? It makes bulk loading repeatable and easy to prepare.

> **Simple example:** A **Forum.json** file can contain PutRequest entries for the Forum table.

16. Unprocessed Items

What is it? Records that were not successfully written during a batch write.

Purpose: Show which records did not get loaded.

Why use it? Checking them helps confirm whether a bulk data load succeeded.

> **Simple example:** If unprocessed items appear, investigate the error and retry.

17. Transaction

What is it? A group of database operations that must all succeed; if one operation fails, none of the changes are applied.

Purpose: Keep related changes consistent across DynamoDB tables.

Why use it? It prevents one related update from succeeding while another fails.

> **Simple example:** Add a comment and increase the related forum's comment count together.

18. transact-write-items

What is it? The AWS CLI DynamoDB command used to perform multiple write/update operations as one transaction.

Purpose: Apply related changes together.

Why use it? It is useful when consistency across multiple tables matters.

> **Simple example:** One transaction can **Put** a Comment and **Update** the Forum count.

19. Wait Command

What is it? An AWS CLI command that waits until a resource reaches a required condition.

Purpose: Make sure a resource is ready before dependent commands run.

Why use it? It helps prevent commands from failing because a table is not ready yet.

Simple example: Use **aws dynamodb wait table-exists --table-name ContentCatalog**.

DynamoDB vs Relational Database — Quick Understanding

DynamoDB / NoSQL	Relational Database
Flexible item attributes.	Defined row/column structure.
Partition/sort keys are central to efficient access.	SQL is useful for complex queries.
Good when flexible data and fast key-based access matter.	Useful when complex queries and strict structure matter.

Key Project Commands

```
aws dynamodb get-item --table-name ContentCatalog --key '{"Id":{"N":"201"}}'
```

```
aws dynamodb get-item --table-name ContentCatalog --key '{"Id":{"N":"202"}}' --projection-expression "Title, ContentType, Services" --return-consumed-capacity TOTAL
```

```
aws dynamodb batch-write-item --request-items file://ContentCatalog.json
```

```
aws dynamodb wait table-exists --table-name ContentCatalog
```

```
aws dynamodb delete-table --table-name ContentCatalog
```

Quick Revision Checklist

• DynamoDB = NoSQL database.

• Table contains items; items contain attributes.

• Partition key = required key used to organize/find data.

• Sort key = optional key for further organizing related data.

• Query design depends heavily on key design.

• CloudShell = browser-based terminal; AWS CLI is already installed.

• get-item = retrieve one item using its key.

• Projection expression = return only selected attributes.

• Strong consistency = latest version; eventual consistency = default/lower-cost option described in the project.

• Transaction = all operations succeed, or none are applied.

• Delete unused DynamoDB resources to avoid unnecessary charges.

Memory trick: Table → Item → Attributes | Partition Key → Find | Sort Key → Narrow/Order related data | CloudShell → Terminal | CLI → Commands | Transaction → All-or-nothing